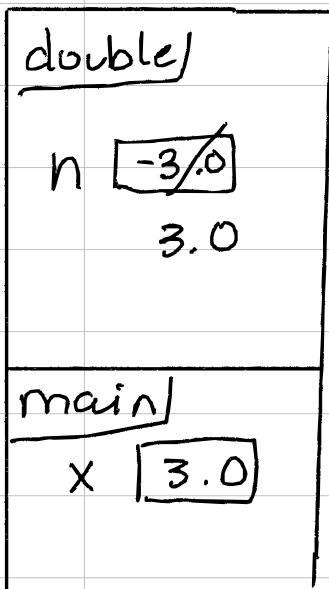


Start with easiest
stack diagram

```
public static double abs(double n) {  
    if (n < 0) {  
        return -n;  
    }  
    return n;  
}
```



double x = abs(-3.0);

From Week 3 Lab

- where is method declared?
 - ↳ signature
 - ↳ parameters
 - ↳ arguments
 - ↳ return type

* Draw execution stack

Draw Execution Stack

SumTwo(first, second) + third

VS

SumTwo(sumTwo(first, second),
third)

* don't confuse printing &
returning *

Recursion

- main() calls our methods
- methods call other methods
- can methods call ... themselves?

Countdown

```
public static void countdown (int n) {  
    if (n == 0) {  
        print ("Blastoff!")  
        return;  
    }
```

```
}
```

```
    print(n);  
    countdown (n-1);
```

```
}
```

```
countdown(3);
```

What do you think this prints?

countdown(3)

print 3

countdown(2)

print 2

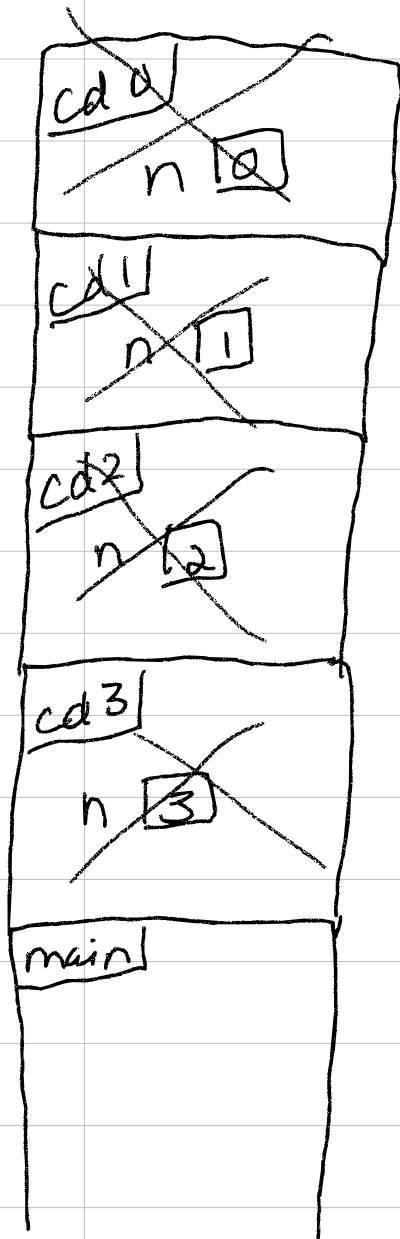
countdown(1)

print 1

countdown(0)

print blastoff!

* each call is a separate
method call



print blast off

print 1

print 2

print 3

Why do we need a base case?

```
public static void cd(int n) {  
    print(n);  
    cd(n-1);  
}
```

cd(3) → what happens?

3 stack overflow!

2

1

0

-1

-2

-3

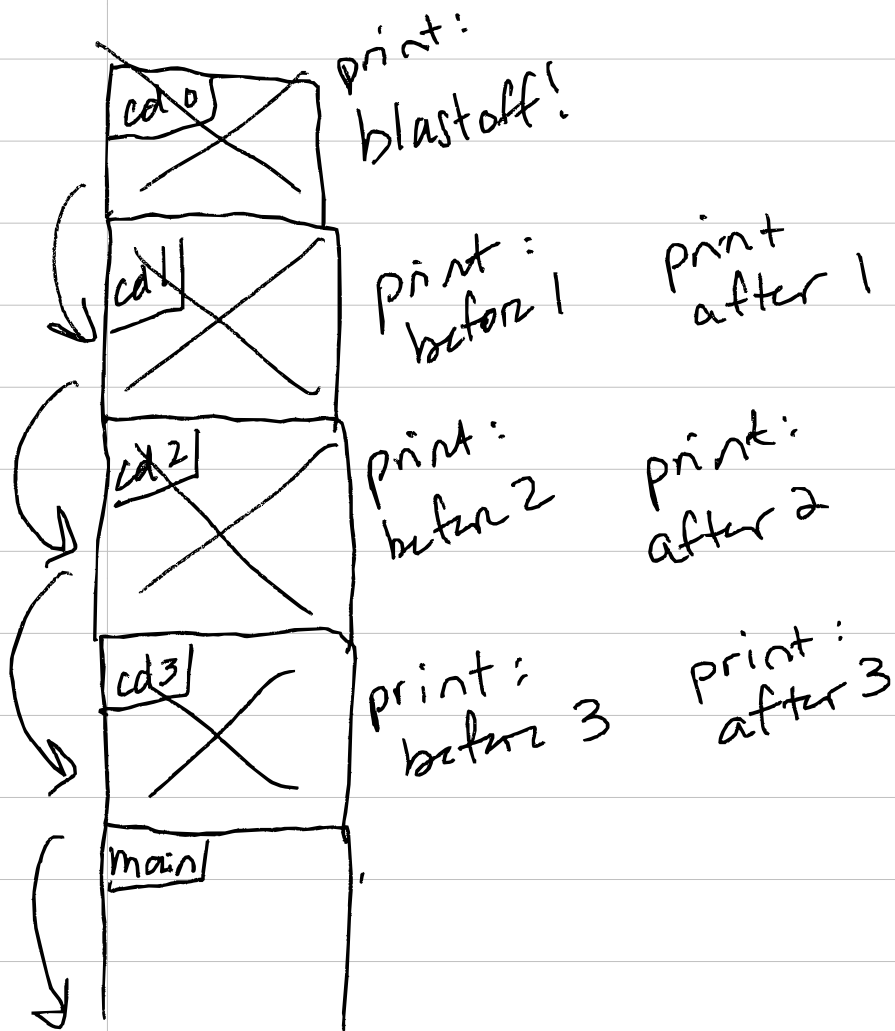
⋮



never
returns!

Recursive Call Returns

```
public static void countdown(int n) {  
    if(n == 0) {  
        print blastoff  
        return  
    }  
    print("before" + n);  
    countdown(n-1)  
    print("after" + n);  
}
```



Returning a value

Let's make the return type int instead of void.

```
public static int sum(int n) {  
    if (n == 1) {  
        return 1;  
    }
```

```
    return n + sum(n-1);
```

```
}
```

* We are summing the #'s
from 1 to n.

$$\text{Sum}(4)$$

$$= 4 + \text{sum}(3)$$

$$= 4 + 3 + \text{sum}(2)$$

$$= 4 + 3 + 2 + \text{sum}(1)$$

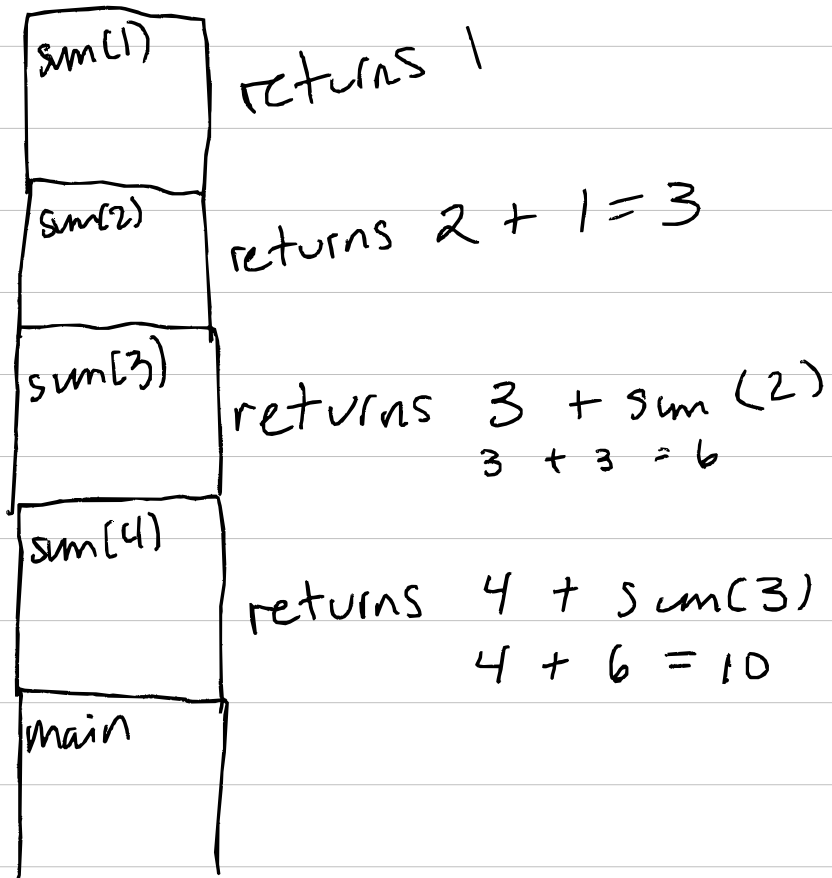
$$= 4 + 3 + 2 + 1$$

$$\text{sum}(1) = 1$$

$$\text{sum}(2) = 2 + 1 = 3$$

$$\text{sum}(3) = 3 + 3 = 6$$

$$\text{sum}(4) = 4 + 6 = 10$$



Recipe for Recursion

1. Identify the simplest possible input.
2. write the answer for that case (base case)
3. figure out how to turn the problem into a smaller version of itself.
(recursive case)
4. Make sure the recursive call moves toward the base case.

Print Stars

public static void printStars(int n)

→ Simplest input: 0

→ answer for base case:

just return

→ smaller problem:

print("*")

printStars(n-1)

```
public static void printStars(int n) {
```

```
    if (n == 0) {
```

```
        return;
```

```
    }
```

← not print line

```
    System.out.print("*");
```

```
    printStars(n-1);
```

```
}
```

print Stars(4) → *

print Stars(3) → **

print Stars(2) → ***

print Stars(1) → ****

Factorial

base case : $n == 1$

recursive case : $n \times \text{fact}(n-1)$

```
public static int factorial(int n) {  
    if (n == 1) {  
        return 1;  
    }  
    return n * factorial(n-1);  
}
```

factorial(4)

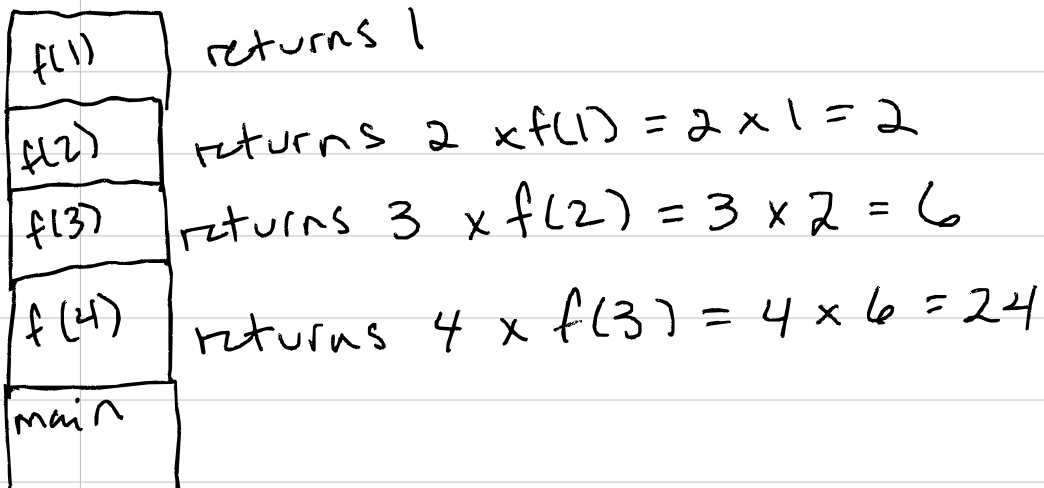
$4 \times \text{fact}(3)$

$4 \times 3 \times \text{fact}(2)$

$4 \times 3 \times 2 \times \text{fact}(1)$

$4 \times 3 \times 2 \times 1$

$= 24$



Countdown Revisited

Recursion : one way to express repetition

Loops are another

- draw stack diagram again for recursive version
- then show the iterative version
- and do its stack diagram